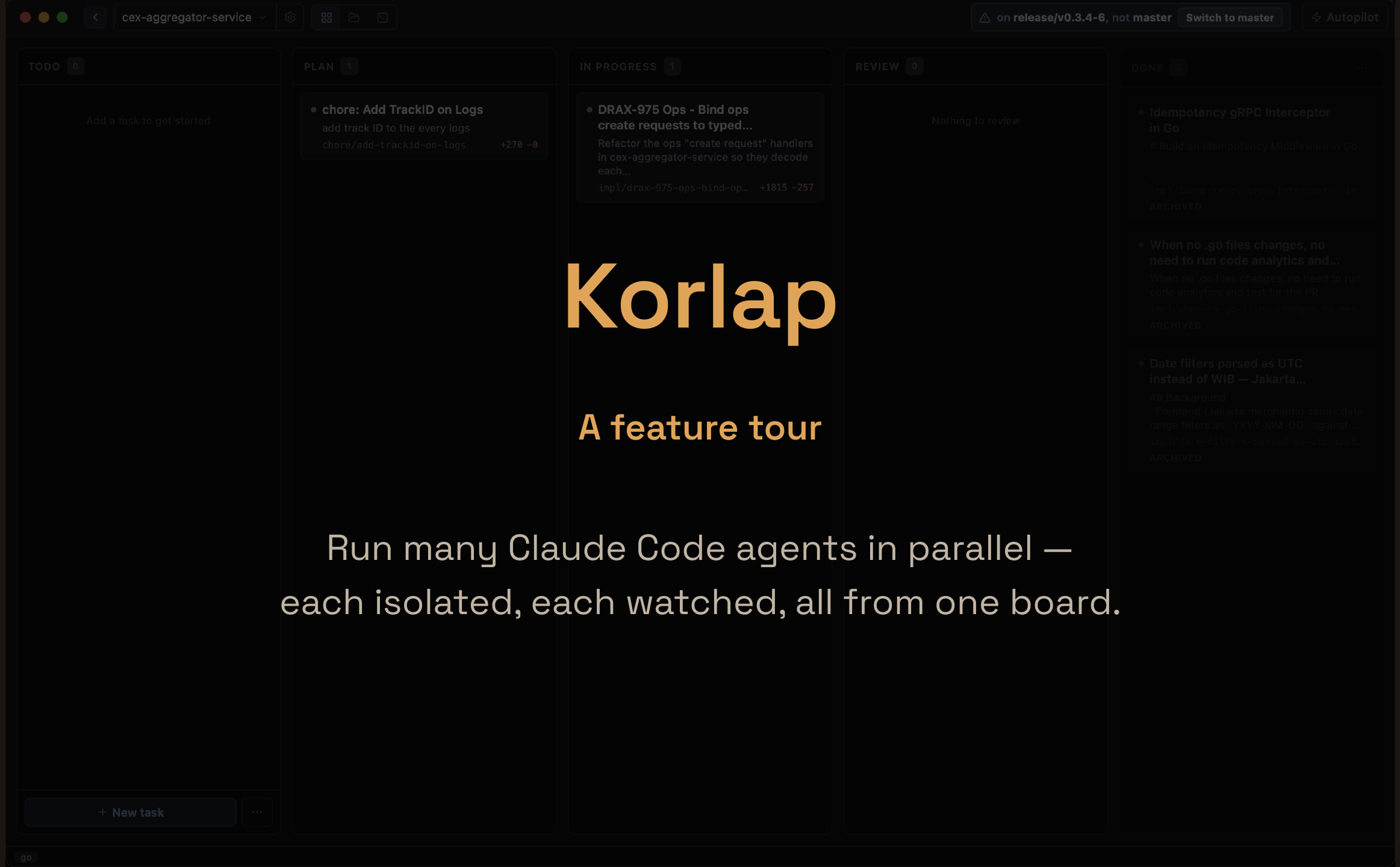


Korlap

A feature tour

Run many Claude Code agents in parallel —
each isolated, each watched, all from one board.



A quick note — this is a fork

What you're seeing is **my fork** of Korlap, where I experiment with my own workflow.

For the canonical project, full docs, and releases, **refer to the original repository**.

- **Original:** [ariaghora/korlap](#)
- **My fork:** [arham09/korlap](#)

Why I use this — the problem it solves

Running several Claude Code agents at once, the plain tooling fought me at every step.

Before	With Korlap
Parallel work collides. One checkout, one branch at a time — stash, switch, stash again. Two tasks can't truly run side by side.	A git worktree per task. Every agent gets its own working tree + branch. Run many at once; none ever steps on another.
A terminal jungle. One <code>claude</code> per tmux pane, N tabs open, no idea which is mid-task and which is waiting on me.	One board, every agent. Switch instantly; status dots show who's running vs. waiting — no terminal-tab roulette.
git diff is hard to read. Raw, unscoped, no highlighting — tough to see what an agent actually changed.	A real diff view. Per-workspace, against the merge-base, syntax-highlighted, live — click a hunk to quote it into chat.

The point: isolation + orchestration + visibility — the three things a wall of terminals can't give you.

What's in the box

Run agents

- Parallel isolated workspaces
- Kanban-driven lifecycle
- Structured chat
- Diff viewer
- Terminal · Files · Editor

Automate & integrate

- LSP for agents
- Branch sync
- `gh` profiles per repo
- Isolated workspaces

Parallel isolated workspaces

Every agent gets its own full repo copy — a `git worktree` on its own branch.

- Agents never collide: separate working tree, branch, diff, and chat history.
 - Each runs an isolated `claude` subprocess with its own session.
 - Confined to their worktree — they can't touch other workspaces or the main repo.
 - Switching between them is instant; each stays alive in the background.
-

How: create a task → Korlap spins up the worktree, branch, and agent automatically.

The sidebar — many agents, each its own workspace

The screenshot displays a web application interface with a sidebar on the left, a main workspace in the center, and a chat window on the right.

Sidebar: The sidebar contains a list of agents under the heading "PLAN 1". The agents are:

- chore: Add TrackID on Logs (chore/add-trackid-on-logs)
- IN PROGRESS 1
 - DRAX-975 Ops - Bind ops cr... (impl/drax-975-ops-bind-ops-crea...)
- REVIEW 0
- DONE 3

Main Workspace: The main workspace shows a "Test Report" for the "DRAX-975 Ops - Bind ops create requests to typed-dtos" branch. The report includes a "Summary" section stating "PASS. All tests pass, lint is clean, and swagger checks pass. The AC coverage matrix is fully exercised. One coverage gap exists in requestToService (the unreachable default: switch branch at lines 1117-1122) — this branch is architecturally unreachable from the 16 typed handler call sites and cannot be tested without exposing the unexported generic function. All other changed functions are at 100%. No tests were added or modified during this validation phase; the implementation's tests are correct."

The "Context used" section lists the following files:

- .agent/tasks/task-awdmAH/task.md
- .agent/tasks/task-awdmAH/context_packet.md
- .agent/tasks/task-awdmAH/implementation_plan.md
- .agent/tasks/task-awdmAH/handoff.md
- CLAUDE.md (testing conventions, 100% coverage requirement, table-driven suite pattern)

The "Files inspected" section lists the following files:

- /Users/arhamawalabiyani/Documents/korlap/workspaces/drax-975-ops-bind-ops-create-requests-to-typed-dtos/internal/ops/http_handler/dto/ops_request_payloads.go — OpsRequestBody int, envelopeToCreateOpsRequest, 16 ToCreateOpsRequest() methods, CreateRetriggerDi, refund example fixed
- /Users/arhamawalabiyani/Documents/korlap/workspaces/drax-975-ops-bind-ops-create-requests-to-typed-dtos/internal/ops/http_handler/ops_handler.go — generic free functions requestToService, parseAndValidateRequest[T] (line 1145); 16 typed call sites; retry-disburse @Param updated
- /Users/arhamawalabiyani/Documents/korlap/workspaces/drax-975-ops-bind-ops-create-requests-to-typed-dtos/internal/ops/http_handler/ops_handler_test.go — reconciled fixtures; 5 new endpoints
- /Users/arhamawalabiyani/Documents/korlap/workspaces/drax-975-ops-bind-ops-create-requests-to-typed-dtos/internal/ops/http_handler/dto/ops_request_payloads_test.go — new table-driven tests, ToCreateOpsRequest() round-trips

The "Tests added or updated" section states: "No tests were added or updated during this validation phase. The implementation's test coverage was 100%. No gaps were found in the changed code paths that required additional tests."

The "Commands run" section shows the following commands:

```
go build ./internal/ops/...
go test ./internal/ops/http_handler/... -cover -count=1
go test ./internal/ops/http_handler/... -coverprofile=/tmp/drax975_cover.out -count=1 && go tool cover -fun
```

Chat Window: The chat window shows a conversation with an agent named "awdmAH". The agent's message is:

awdmAH/ (task.md , context_packet.md , implementation_plan.md , handoff.md , test_report.md , implementation_review.md). Review verdict: Pass. Ready for you to commit/PR.

The user's response is:

- /agent-task : leave disable-model-invocation: true . To run the real loop next time, invoke it standalone (/agent-task ... on its own), not nested in another command's args.

The agent's response is:

Nothing outstanding on my end. What would you like to do next?

The chat window also includes a "Yes" button, a "No" button, and a text input field with the placeholder "Ask to make changes, @mention files". The input field contains the text "Thinking".

A kanban that drives the agents

The board **is** the lifecycle. Dragging a card advances the agent through:

- **Todo** → defined, no agent yet.
- **Plan** → the agent designs the change (plan mode, optional `pLan`).
- **In Progress** → full-permission implementation.
- **Review** → a PR is open; keep iterating.
- **Done** → merged; worktree kept for audit.

Status dots: pulsing amber = agent running · olive = waiting on you.

The board — cards flow Todo → Done

The screenshot displays a Kanban board for the project "cex-aggregator-service". The board is organized into five columns: TODO (0 cards), PLAN (1 card), IN PROGRESS (1 card), REVIEW (0 cards), and DONE (3 cards). A detailed view of the task in the IN PROGRESS column is shown, titled "DRAX-975 Ops - Bind ops create requests to typed DTOs".

Task Details:

- Category:** chore: Add TrackID on Logs
- Sub-tasks:** add track ID to the every logs, chore/add-trackid-on-logs
- Status:** WAITING
- Description:** Refactor the ops "create request" handlers in cex-aggregator-service so they decode each request body into its typed DTO at runtime, instead of the generic `dto.CreateOpsRequest (json.RawMessage)`. This makes the typed wrappers real (validated + drift-proof), not documentation-only. Ticket: [DRAX-XXX].
- Context:**
 - Handlers: `internal/ops/http_handler/ops_handler.go`. Every create endpoint calls `requestToService(ctx, requestType) -> parseAndValidateRequest`, which decodes into `dto.CreateOpsRequest` and stores the raw payload in `entity.OpsRequest.RequestPayload`.
 - Typed wrappers already exist: `internal/ops/http_handler/dto/ops_request_payloads.go` (15 of them). Their `RequestPayload` is the exact `service.XxxRequest` struct. They are already referenced by the `@Param` swagger annotations.
 - Services keep the signature `Request(ctx, entity.OpsRequest);` the concrete payload is unmarshalled later in each service's `Execute()`. Do NOT change service signatures.
- Goal:**
 - Decode each endpoint's body into its typed wrapper and validate the full body (payload included) with `shared.Validate`, returning 400 before storing on any failure.
 - Re-marshal the typed payload into `entity.OpsRequest.RequestPayload` so the stored shape and the service layer are unchanged.
 - Add `CreateRetriggerDisburseOpsRequest` (payload

Branch and Changes:

- Branch: `impl/drax-975-ops-bind-ops-create-requests-to...`
- Changes: +1815 -257

Tip: +click card to open directly

Buttons: Go to workspace →

Other Cards in the Board:

- TODO:** Add a task to get started
- PLAN:** chore: Add TrackID on Logs
- IN PROGRESS:** DRAX-975 Ops - Bind ops create requests to typed DTOs
- REVIEW:** Nothing to review
- DONE:**
 - Idempotency gRPC Interceptor in Go
 - When no .go files changes, no need to run code analytics and...
 - Date filters parsed as UTC instead of WIB — Jakarta...

Structured chat

Agent output is **parsed, not dumped** — a rich message list, not a raw log.

- From `stream-json`: tool calls, thinking, and results stay distinct.
 - Queue messages while the agent is busy.
 - Paste **images**, mention `@file` paths, run **slash commands**.
 - Per-message token counts; full history persisted per workspace.
-

Where: the Chat tab in any workspace.

Structured chat — tool calls, results, tokens

The screenshot shows a code editor interface with a sidebar on the left and a main chat area on the right. The sidebar contains a list of tasks under the heading "PLAN 1". The main chat area displays a "Test Report" for a specific task, including a summary, chat history, and code snippets.

PLAN 1

- chore: Add TrackID on Logs
chore/add-trackid-on-logs
- IN PROGRESS 1
 - DRAX-975 Ops - Bind ops cr...
impl/drax-975-ops-bind-ops-crea...
- REVIEW 0
- DONE 3

Test Report

Summary

Chat

5 +
6 +
7 +

Tests pass (246), but `normalizeJSON` now unused → lint fail. Check + remove.

Unused. Remove it.

1 - func normalizeJSON(raw json.RawMessage) json.RawMessage {
2 - var tmp interface{}
3 - _ = json.Unmarshal(raw, &tmp)
4 - b, _ := json.Marshal(tmp)
5 - return b
6 - }
7 -
8 -
1 +

1 - func normalizeJSON(raw json.RawMessage) json.RawMessage {
2 - var tmp interface{}
3 - _ = json.Unmarshal(raw, &tmp)
4 - b, _ := json.Marshal(tmp)
5 - return b
6 - }
7 -

Ask to make changes, @mention files

Thinking Plan Default (Opus) Claude

go test ./internal/ops/http handler/... -coverprofile=/tmp/drax975 cover.out -count=1 && go tool cover -fun

Diff viewer

See exactly what each agent changed — **against the merge-base**, syntax-highlighted.

- Unified diff scoped to the workspace's branch.
- Click a hunk header to **quote it back into chat** as context.
- Updates live as the agent edits.

Where: the Diff tab.  runs an AI Review of the same diff.

Diff against the merge-base, syntax-highlighted

PLAN 1

chore: Add TrackID on Logs
chore/add-trackid-on-logs

IN PROGRESS 1

DRAX-975 Ops - Bind ops cr...
impl/drax-975-ops-bind-ops-crea...

REVIEW 0

DONE 3

Diff +1815 -257 Files Docs impl/drax-975-ops-bind-ops-create-requests-to-typed-dtos master

Changes 11 +1815 -257

+ .gitignore +5 -0

~ docs/merchant-api/doc... +37 -13

~ docs/merchant-api/swa... +37 -13

~ docs/merchant-api/swag... +28 -9

~ docs/ops-api/docs.go +44 -10

~ docs/ops-api/swagger.y... +44 -10

~ docs/ops-api/swagger.y... +32 -8

~ internal/ops/http_han... +100 -2

~ internal/ops/http_han... +30 -25

~ internal/ops/http_... +1323 -167

+ internal/ops/http_han... +135 -0

@@ -421,7 +421,7 @@ func (handler *OpsHandler) RequestMerchantWebhook(ctx *fiber.Ctx) error {

421 421 // @Failure 500 {object} shared.ResponseError

422 422 // @Router /ops/v1/requests/merchant-registration-va [post]

423 423 func (handler *OpsHandler) RequestMerchantRegistrationVA(ctx *fiber.Ctx) error {

424 - return handler.requestToService(ctx, entity.MerchantRegistrationVARequestType)

424 + return requestToService(dto.CreateMerchantRegistrationVAOpsRequest)(handler, ctx, entity.MerchantRegistrationVARequestType)

425 425 }

426 426 }

427 427 // RequestMerchantIpWhitelist godoc

@@ -438,7 +438,7 @@ func (handler *OpsHandler) RequestMerchantRegistrationVA(ctx *fiber.Ctx) error {

438 438 // @Failure 500 {object} shared.ResponseError

439 439 // @Router /ops/v1/requests/merchant-ip-whitelist [post]

440 440 func (handler *OpsHandler) RequestMerchantIpWhitelist(ctx *fiber.Ctx) error {

441 - return handler.requestToService(ctx, entity.MerchantIpWhitelistRequestType)

441 + return requestToService(dto.CreateMerchantIPWhitelistOpsRequest)(handler, ctx, entity.MerchantIpWhitelistRequestType)

442 442 }

443 443 }

444 444 // RequestRetriggerFiatWithdrawal godoc

@@ -455,7 +455,7 @@ func (handler *OpsHandler) RequestMerchantIpWhitelist(ctx *fiber.Ctx) error {

455 455 // @Failure 500 {object} shared.ResponseError

456 456 // @Router /ops/v1/requests/fiat-withdrawal-recovery [post]

457 457 func (handler *OpsHandler) RequestRetriggerFiatWithdrawal(ctx *fiber.Ctx) error {

458 - return handler.requestToService(ctx, entity.RetriggerFiatWithdrawalRequestType)

458 + return requestToService(dto.CreateFiatWithdrawalRecoveryOpsRequest)(handler, ctx, entity.RetriggerFiatWithdrawalRequestType)

459 459 }

460 460 }

461 461 // RequestMerchantFeaturesConfig godoc

@@ -472,7 +472,7 @@ func (handler *OpsHandler) RequestRetriggerFiatWithdrawal(ctx *fiber.Ctx) error {

472 472 // @Failure 500 {object} shared.ResponseError

473 473 // @Router /ops/v1/requests/merchant-features-config [post]

474 474 func (handler *OpsHandler) RequestMerchantFeaturesConfig(ctx *fiber.Ctx) error {

475 - return handler.requestToService(ctx, entity.MerchantFeaturesRequestType)

475 + return requestToService(dto.CreateMerchantFeaturesConfigOpsRequest)(handler, ctx, entity.MerchantFeaturesRequestType)

476 476 }

477 477 }

478 478 // RequestRetriggerSwap godoc

@@ -489,7 +489,7 @@ func (handler *OpsHandler) RequestMerchantFeaturesConfig(ctx *fiber.Ctx) error {

489 489 // @Failure 500 {object} shared.ResponseError

490 490 // @Router /ops/v1/requests/swap-recovery [post]

491 491 func (handler *OpsHandler) RequestRetriggerSwap(ctx *fiber.Ctx) error {

90

Chat ^

Terminal, Files & Editor

Everything you'd reach for, inside the workspace — no context-switch to another app.

Terminal

Raw `zsh` PTY tabs, one or many, rooted in the worktree directory. Full native terminal via `xterm.js`.

Files & Editor

Browse the worktree as a tree; open and edit any file in a CodeMirror 6 editor with multi-language highlighting.

Tip: the agent owns the worktree — edit by hand sparingly.

LSP — agents that navigate code

A per-repo **language-server pool**, exposed to the agent over MCP.

- Built-in defaults for Rust, TypeScript, Svelte, Go, and more — plus custom servers.
- The agent can **hover, go-to-definition, find-references, rename, and read diagnostics** across the whole worktree.
- Far cheaper and more precise than grepping for symbols.

Where: configure & monitor servers in , → LSP.

Branch sync

Keep long-running branches current without leaving the app.

- The header shows **how many commits you're behind** the base branch.
- One click merges the base in (`git merge origin/<base>` inside the worktree).

Trigger: `⌘U` — Update branch from base.

Tailor each task & each repo

Per task

- Plan-mode & Thinking toggles
- **Model override** (Sonnet / Opus / Haiku)
- `@file` mentions & pasted images
- "Add and Start" to spawn instantly

Per repo `⌘,`

- `system_prompt`, default model
- `default_start_phase`, `default_plan`
- `openspec_enabled`,

**You orchestrate. Agents
execute.**